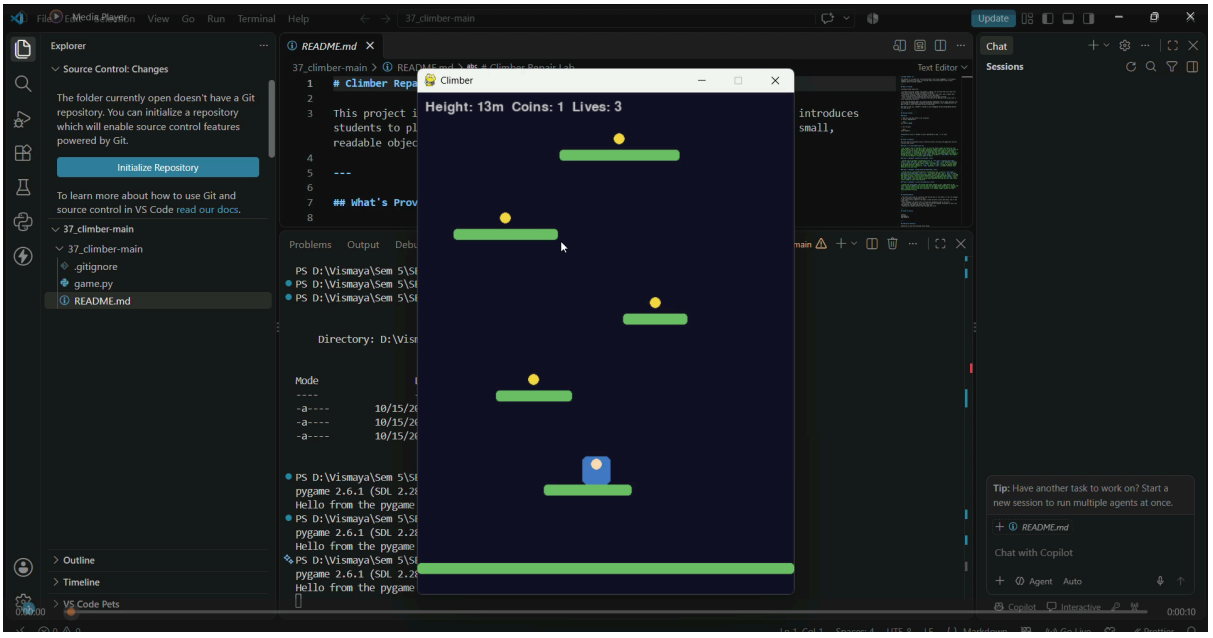
Software Engineering
Lab 4: VibeCoding

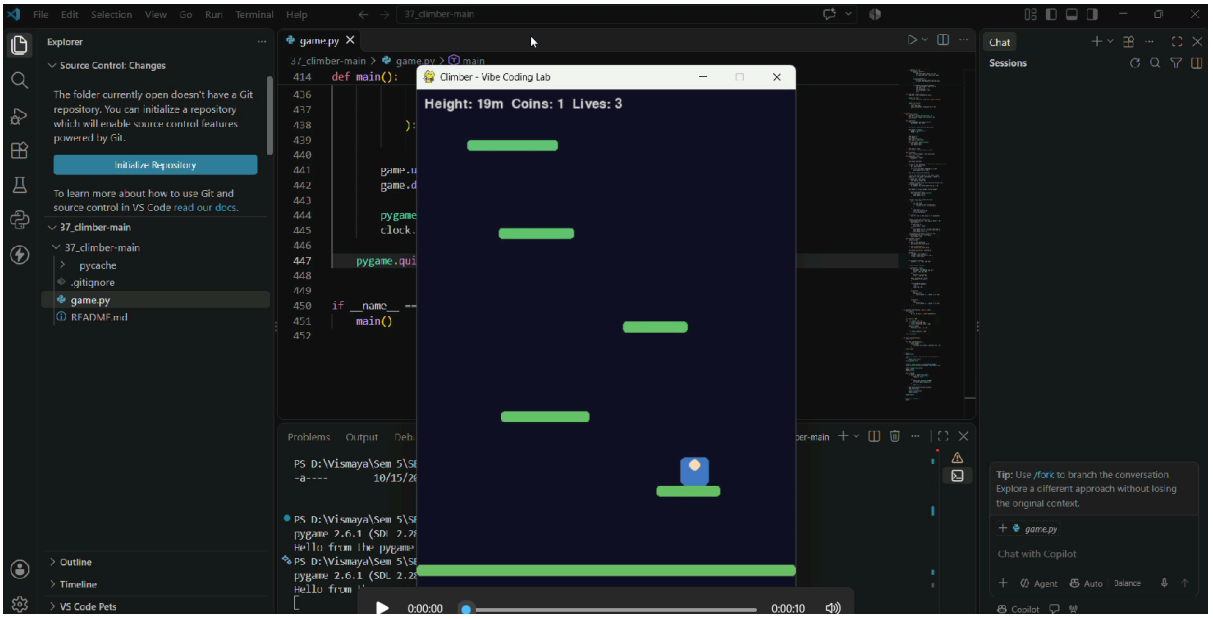
Submitted by:
Name: Vismaya Harish
SRN: PES2UG24CS598
Class: 5 J

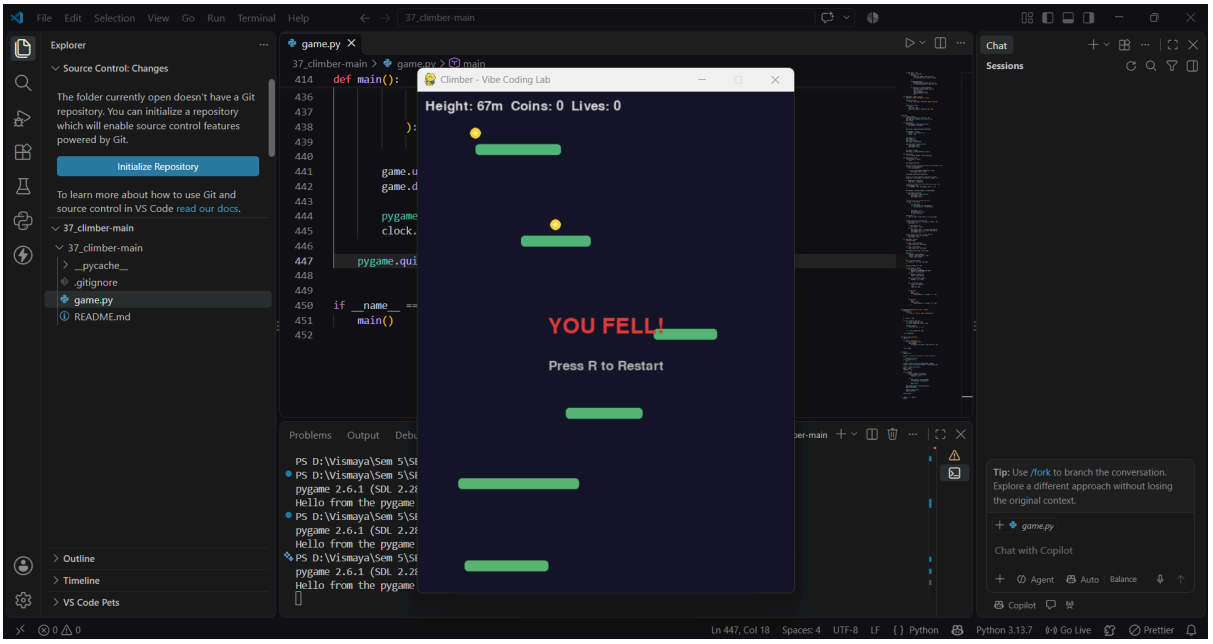
Git Repo Link:
https://github.com/VismayaHarish/SE_Lab4_climber.git
Chat History Link:
https://chatgpt.com/share/6ad097e7-6f80-83ee-a011-3a88a5cfd55c

Before:



After:





Updated Game.py Code:

```
import random
import math
import array
import pygame

WIDTH, HEIGHT = 480, 640
FPS = 60
BG = (20, 20, 40)
```

```
GRAVITY = 0.5
JUMP_SPEED = -13
MOVE_SPEED = 4

PLAYER_W, PLAYER_H = 36, 36
PLATFORM_H = 14
COIN_R = 7
LIVES_START = 3


# FEATURE 1: Platform colours gradually change as the player climbs.
def platform_color(index, total):
    if total <= 0:
        return (100, 180, 100)

    progress = max(0.0, min(1.0, index / total))

    if progress < 0.5:
        t = progress * 2
        start = (100, 180, 100)
        end = (40, 190, 180)
    else:
        t = (progress - 0.5) * 2
        start = (40, 190, 180)
        end = (170, 90, 210)

    return tuple(
        int(start[i] + (end[i] - start[i]) * t)
        for i in range(3)
    )


# FEATURE 2: Every third elevated platform moves horizontally.
def moving_platform_speed(index, total):
    if index <= 0 or index % 3 != 0:
        return 0

    progress = index / max(1, total)
    return 1.0 + min(0.6, progress * 0.6)


# FEATURE 3: Play a short synthesized sound when a coin is collected.
def on_coin_collected():
```

```
    try:
        if not pygame.mixer.get_init():
            return

        if not hasattr(on_coin_collected, "_sound"):
            sample_rate = 22050
            duration = 0.08
            count = int(sample_rate * duration)
            samples = array.array("h")

            for i in range(count):
                t = i / sample_rate
                envelope = max(0.0, 1.0 - i / count)
                frequency = 880 if t < duration / 2 else 1175
                value = int(
                    7000 * envelope *
                    math.sin(2 * math.pi * frequency * t)
                )
                samples.append(value)

            on_coin_collected._sound = pygame.mixer.Sound(
                buffer=samples.tobytes()
            )

        on_coin_collected._sound.play()

    except (pygame.error, RuntimeError, AttributeError):
        pass


class Platform:
    def __init__(self, index, total, x, y, w=120, movable=True):
        self.rect = pygame.Rect(x, y, w, PLATFORM_H)

        self.speed = (
            moving_platform_speed(index, total)
            if movable else 0
        )

        self.bounds = (
            max(0, x - 60),
            min(WIDTH - w, x + 60)
        )
```

```
        self.color = platform_color(index, total)

    def update(self):
        if not self.speed:
            return 0

        dx = self.speed

        if self.rect.x <= self.bounds[0] and self.speed < 0:
            self.speed = abs(self.speed)
        elif self.rect.x >= self.bounds[1] and self.speed > 0:
            self.speed = -abs(self.speed)

        dx = self.speed
        self.rect.x = max(
            self.bounds[0],
            min(self.bounds[1], self.rect.x + dx)
        )

        return dx

    def draw(self, screen, cam_y):
        pygame.draw.rect(
            screen,
            self.color,
            self.rect.move(0, -cam_y),
            border_radius=4
        )


class Coin:
    def __init__(self, x, y):
        self.pos = pygame.Vector2(x, y)
        self.taken = False

    def draw(self, screen, cam_y):
        coin_position = (
            int(self.pos.x),
            int(self.pos.y - cam_y)
        )

        pygame.draw.circle(
```

```
            screen, (250, 210, 60), coin_position, COIN_R
        )
        pygame.draw.circle(
            screen, (255, 240, 150), coin_position, 3
        )


class Player:
    def __init__(self, x, y):
        self.rect = pygame.Rect(x, y, PLAYER_W, PLAYER_H)
        self.vel_y = 0.0
        self.vel_x = 0
        self.on_ground = False
        self.standing_on = None
        self.color = (60, 120, 200)

    def move(self, keys):
        self.vel_x = 0

        if keys[pygame.K_LEFT] or keys[pygame.K_a]:
            self.vel_x = -MOVE_SPEED

        if keys[pygame.K_RIGHT] or keys[pygame.K_d]:
            self.vel_x = MOVE_SPEED

        if (
            keys[pygame.K_SPACE]
            or keys[pygame.K_w]
            or keys[pygame.K_UP]
        ) and self.on_ground:
            self.vel_y = JUMP_SPEED
            self.on_ground = False

    def update(self, platforms):
        self.vel_y = min(self.vel_y + GRAVITY, 12)

        previous_bottom = self.rect.bottom

        self.rect.x = max(
            0,
            min(
                WIDTH - self.rect.width,
                self.rect.x + self.vel_x
```

```
            )
        )

        self.rect.y += int(self.vel_y)

        self.on_ground = False
        self.standing_on = None

        if self.vel_y >= 0:
            for plat in platforms:
                overlaps = (
                    self.rect.right > plat.rect.left
                    and self.rect.left < plat.rect.right
                )

                crossed_platform = (
                    previous_bottom <= plat.rect.top + 1
                    and self.rect.bottom >= plat.rect.top
                )

                if overlaps and crossed_platform:
                    self.rect.bottom = plat.rect.top
                    self.vel_y = 0
                    self.on_ground = True
                    self.standing_on = plat
                    break

    def draw(self, screen, cam_y):
        draw_rect = self.rect.move(0, -cam_y)

        pygame.draw.rect(
            screen, self.color, draw_rect, border_radius=6
        )

        pygame.draw.circle(
            screen,
            (255, 220, 180),
            (draw_rect.centerx, draw_rect.top + 10),
            7
        )


class Game:
```

```
    def __init__(self):
        self.font = pygame.font.Font(None, 26)
        self.big_font = pygame.font.Font(None, 42)
        self.reset()

    def reset(self):
        self.platforms = generate_platforms(
            60, HEIGHT - 40, WIDTH
        )

        self.coins = spawn_coins(self.platforms)

        self.player = Player(
            WIDTH // 2 - PLAYER_W // 2,
            HEIGHT - 100
        )

        self.cam_y = 0
        self.height = 0
        self.coin_score = 0
        self.lives = LIVES_START

        self.last_safe = pygame.Vector2(
            self.player.rect.x,
            self.player.rect.y
        )

        self.state = "play"
        self.top_y = self.platforms[-1].rect.y

    def score(self):
        return int(self.height) + self.coin_score

    def update(self, keys):
        if self.state != "play":
            return

        self.player.move(keys)

        # Move platforms and carry the player when standing on one.
        for plat in self.platforms:
            dx = plat.update()
```

```
            if dx and self.player.standing_on is plat:
                self.player.rect.x += dx

        self.player.update(self.platforms)

        # Scroll the camera upwards as the player climbs.
        target_cam = self.player.rect.centery - HEIGHT // 2

        if target_cam < self.cam_y:
            self.cam_y = target_cam

        # FEATURE 4: Height score never decreases during a run.
        current_height = max(
            0, (HEIGHT - 40 - self.player.rect.y) // 10
        )

        self.height = max(self.height, current_height)

        if self.player.on_ground:
            self.last_safe = pygame.Vector2(
                self.player.rect.x,
                self.player.rect.y
            )

        # Collect coins and increase the score.
        for coin in self.coins:
            if (
                not coin.taken
                and self.player.rect.collidepoint(
                    int(coin.pos.x), int(coin.pos.y)
                )
            ):
                coin.taken = True
                self.coin_score += 50
                on_coin_collected()

        self.coins = [
            coin for coin in self.coins if not coin.taken
        ]

        # Handle falling and losing lives.
        if self.player.rect.top - self.cam_y > HEIGHT + 50:
            self.lives -= 1
```

```
            if self.lives <= 0:
                self.state = "lose"
            else:
                self.player.rect.x = int(self.last_safe.x)
                self.player.rect.y = int(self.last_safe.y)
                self.player.vel_y = 0

        # Check whether the player reached the top.
        if self.player.rect.y <= self.top_y:
            self.state = "win"

    def draw(self, screen):
        screen.fill(BG)

        for plat in self.platforms:
            plat.draw(screen, self.cam_y)

        for coin in self.coins:
            coin.draw(screen, self.cam_y)

        self.player.draw(screen, self.cam_y)

        hud_text = (
            f"Height: {self.height}m  "
            f"Coins: {self.coin_score // 50}  "
            f"Lives: {self.lives}"
        )

        hud = self.font.render(
            hud_text, True, (220, 220, 220)
        )

        screen.blit(hud, (10, 10))

        if self.state != "play":
            if self.state == "win":
                message = "YOU REACHED THE TOP!"
                color = (80, 220, 80)
            else:
                message = "YOU FELL!"
                color = (220, 60, 60)
```

```
            msg = self.big_font.render(
                message, True, color
            )

            sub = self.font.render(
                "Press R to Restart",
                True,
                (180, 180, 180)
            )

            screen.blit(
                msg,
                msg.get_rect(
                    center=(WIDTH // 2, HEIGHT // 2 - 20)
                )
            )

            screen.blit(
                sub,
                sub.get_rect(
                    center=(WIDTH // 2, HEIGHT // 2 + 30)
                )
            )


def generate_platforms(num, start_y, width):
    platforms = [
        Platform(
            0, num, 0, start_y, width, movable=False
        )
    ]

    y = start_y - 100

    for i in range(1, num + 1):
        w = random.randint(80, 160)
        x = random.randint(20, width - 140)

        platforms.append(
            Platform(i, num, x, y, w)
        )

        y -= random.randint(70, 120)
```

```
    return platforms


def spawn_coins(platforms):
    coins = []

    for plat in platforms[1:]:
        if random.random() < 0.4:
            coins.append(
                Coin(plat.rect.centerx, plat.rect.top - 14)
            )

    return coins


def main():
    pygame.init()

    # Audio is optional; the game still works without it.
    try:
        pygame.mixer.init()
    except pygame.error:
        pass

    screen = pygame.display.set_mode((WIDTH, HEIGHT))
    pygame.display.set_caption("Climber - Vibe Coding Lab")

    clock = pygame.time.Clock()
    game = Game()
    running = True

    while running:
        for event in pygame.event.get():
            if event.type == pygame.QUIT:
                running = False

            elif (
                event.type == pygame.KEYDOWN
                and event.key == pygame.K_r
            ):
                game.reset()
```

```
        game.update(pygame.key.get_pressed())
        game.draw(screen)

        pygame.display.flip()
        clock.tick(FPS)

    pygame.quit()


if __name__ == "__main__":
    main()
```